

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №11.

Тема: «»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 19

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи

Операции над односвязным списком:

1. Добавление элемента в конец (с использованием одного указателя).
2. Добавление элемента в конец (с использованием указателей).
3. Добавление элемента в начало.
4. Вставка элемента после позиции с заданным индексом.
5. Вставка элементов после узла с указанным ключом (значением).
6. Удаление первого элемента.
7. Удаление последнего элемента.
8. Удаление элемента, следующего за указанным индексом.
9. Удаление элемента по значению (ключу).
10. Вывод списка на экран.
11. Сохранение списка в файл.
12. Полное удаление (очистка) списка.
13. Восстановление списка из файла.

Анализ задачи

Однонаправленный список (List1)

Это цепочка узлов, где каждый узел хранит значение и указатель на следующий узел. Голова списка — единственная точка входа. Чтобы добавить элемент в начало, создается новый узел, его указатель `next` устанавливается на текущую голову, и голова перенаправляется на новый узел. Чтобы добавить в конец, нужно пройти по всей цепочке до последнего узла и прицепить новый узел к его `next`. Удаление по позиции требует обхода до нужного места, после чего указатель предыдущего узла перепрыгивает через удаляемый узел. Поиск всегда начинается с головы и идет последовательно.

Двунаправленный список с указателями на начало и конец (List2)

Логика та же, что у однонаправленного списка, но дополнительно хранится указатель на последний узел. Это позволяет добавлять элементы в конец без обхода всей цепочки — новый узел просто прицепляется к `last`, после чего `last` сдвигается на него. При удалении последнего элемента нужно корректировать `last`, перемещая его на предыдущий узел. Все остальные операции выполняются так же, как в однонаправленном списке.

Двусвязный список (List3)

Каждый узел хранит два указателя: на следующий и на предыдущий узел. Хранятся указатели на голову и хвост. Добавление в начало и конец выполняется за $O(1)$ благодаря хранению обоих концов. Ключевое отличие — удаление произвольного узла не требует обхода для поиска предыдущего,

так как к нему можно обратиться через prev удаляемого узла. Достаточно соединить prev->next с next и next->prev с prev. Благодаря двунаправленности можно обходить список с конца, что используется в методе printRev.

Очередь (Queue)

Реализована на основе двусвязного списка. Работает по принципу FIFO — первый пришел, первый ушел. Добавление (enq) всегда происходит в конец очереди: новый узел становится новым rear, а старый rear указывает на него. Удаление (deq) всегда происходит из начала: front сдвигается на следующий узел, старый front удаляется. Хранятся указатели на оба конца, что позволяет обе операции выполнять за $O(1)$. Дополнительно реализована вставка перед произвольной позицией, которая нарушает принцип очереди, но демонстрирует возможности базовой структуры.

Стек (Stack)

Реализован на основе однонаправленного списка. Работает по принципу LIFO — последний пришел, первый ушел. Все операции выполняются через вершину стека. Добавление (push) — новый узел становится вершиной, его next указывает на старую вершину. Удаление (pop) — вершина удаляется, а новой вершиной становится следующий узел. Достаточно одного указателя top. При загрузке из файла элементы сначала читаются в прямой список, а затем извлекаются с конца и помещаются в стек, чтобы сохранить порядок строк.

Код

```
#include <iostream>

#include <fstream>

#include <string>

#include <windows.h>

#include <locale>

using namespace std;

struct Node1 {
    string val;
    Node1* next;
    Node1(const string& v) : val(v), next(nullptr) {}
};
```

```
};
```

```
struct Node2 {  
    string val;  
    Node2* prev;  
    Node2* next;  
    Node2(const string& v) : val(v), prev(nullptr), next(nullptr) {}  
};
```

```
struct List1 {  
    Node1* head;  
  
    List1() : head(nullptr) {}  
    ~List1() { clear(); }  
  
    bool empty() const { return head == nullptr; }  
  
    void print() const {  
        if (empty()) {  
            cout << "Список пустой.\n";  
            return;  
        }  
        cout << "Список: ";  
        Node1* p = head;  
        int i = 1;  
        while (p) {  
            cout << "[" << i++ << ": " << p->val << "]" ";  
            p = p->next;  
        }  
        cout << "\n";  
    }  
  
    void pushF(const string& v) {
```

```

Node1* n = new Node1(v);

n->next = head;

head = n;
}

```

```

void pushB(const string& v) {

    Node1* n = new Node1(v);

    if (!head) {

        head = n;

        return;

    }

    Node1* p = head;

    while (p->next) p = p->next;

    p->next = n;

}

```

```

void addFront(int k, const string vals[]) {

    for (int i = k - 1; i >= 0; i--) pushF(vals[i]);

    cout << "Добавлено " << k << " элементов в начало.\n";

}

```

```

bool delPos(int k, const int nums[]) {

    if (empty()) {

        cout << "Список пустой.\n";

        return false;

    }

    int cnt = 0;

    for (int i = 0; i < k; i++) {

        if (delAt(nums[i])) cnt++;

    }

    if (cnt > 0) {

        cout << "Удалено " << cnt << " из " << k << " элементов.\n";

        return true;

    }

}

```

```

    }

    cout << "Ничего не удалено.\n";

    return false;
}

```

```

bool delF() {

    if (!head) return false;

    Node1* t = head;

    head = head->next;

    delete t;

    return true;

}

```

```

bool delB() {

    if (!head) return false;

    if (!head->next) {

        delete head;

        head = nullptr;

        return true;

    }

    Node1* p = head;

    while (p->next->next) p = p->next;

    delete p->next;

    p->next = nullptr;

    return true;

}

```

```

bool delKey(const string& key) {

    if (!head) return false;

    if (head->val == key) return delF();

    Node1* p = head;

    while (p->next && p->next->val != key) p = p->next;

    if (!p->next) return false;

```

```

    Node1* t = p->next;
    p->next = t->next;
    delete t;
    return true;
}

```

```

bool delAt(int pos) {
    if (pos <= 0 || !head) return false;
    if (pos == 1) return delF();
    Node1* p = head;
    int i = 1;
    while (p->next && i < pos - 1) {
        p = p->next;
        i++;
    }
    if (!p->next) return false;
    Node1* t = p->next;
    p->next = t->next;
    delete t;
    return true;
}

```

```

void save(const string& f) const {
    ofstream out(f);
    if (!out) {
        cout << "Ошибка открытия файла.\n";
        return;
    }
    for (Node1* p = head; p; p = p->next) out << p->val << "\n";
    cout << "Сохранено в " << f << "\n";
}

```

```

void load(const string& f) {

```

```

ifstream in(f);

if (!in) {
    cout << "Ошибка открытия файла.\n";
    return;
}

clear();

string s;

while (getline(in, s)) if (!s.empty()) pushB(s);

cout << "Загружено из " << f << "\n";
}

void clear() {
    while (head) {
        Node1* t = head;
        head = head->next;
        delete t;
    }
}

};

struct List2 {
    Node1* first;
    Node1* last;

    List2() : first(nullptr), last(nullptr) {}
    ~List2() { clear(); }

    bool empty() const { return first == nullptr; }

    void print() const {
        if (empty()) {
            cout << "Список пустой.\n";
            return;
        }
    }
};

```



```

    }

    cout << "Список: ";

    Node1* p = first;

    int i = 1;

    while (p) {

        cout << "[" << i++ << ": " << p->val << "]" ";

        p = p->next;

    }

    cout << "\n";

}

```

```

void pushF(const string& v) {

    Node1* n = new Node1(v);

    n->next = first;

    first = n;

    if (!last) last = n;

}

```

```

void pushB(const string& v) {

    Node1* n = new Node1(v);

    if (!first) {

        first = last = n;

        return;

    }

    last->next = n;

    last = n;

}

```

```

void addFront(int k, const string vals[]) {

    for (int i = k - 1; i >= 0; i--) pushF(vals[i]);

    cout << "Добавлено " << k << " элементов в начало.\n";

}

```

```

bool delPos(int k, const int nums[]) {
    if (empty()) {
        cout << "Список пустой.\n";
        return false;
    }
    int cnt = 0;
    for (int i = 0; i < k; i++) {
        if (delAt(nums[i])) cnt++;
    }
    if (cnt > 0) {
        cout << "Удалено " << cnt << " из " << k << " элементов.\n";
        return true;
    }
    cout << "Ничего не удалено.\n";
    return false;
}

```

```

bool delF() {
    if (!first) return false;
    Node1* t = first;
    first = first->next;
    delete t;
    if (!first) last = nullptr;
    return true;
}

```

```

bool delB() {
    if (!first) return false;
    if (first == last) {
        delete first;
        first = last = nullptr;
        return true;
    }
}

```

```

Node1* p = first;
while (p->next != last) p = p->next;
delete last;
last = p;
last->next = nullptr;
return true;
}

```

```

bool delAt(int pos) {
    if (pos <= 0 || !first) return false;
    if (pos == 1) return delF();
    Node1* p = first;
    int i = 1;
    while (p->next && i < pos - 1) {
        p = p->next;
        i++;
    }
    if (!p->next) return false;
    Node1* t = p->next;
    p->next = t->next;
    if (t == last) last = p;
    delete t;
    return true;
}

```

```

void save(const string& f) const {
    ofstream out(f);
    if (!out) {
        cout << "Ошибка открытия файла.\n";
        return;
    }
    for (Node1* p = first; p; p = p->next) out << p->val << "\n";
    cout << "Сохранено в " << f << "\n";
}

```

```
}
```

```
void load(const string& f) {  
    ifstream in(f);  
    if (!in) {  
        cout << "Ошибка открытия файла.\n";  
        return;  
    }  
    clear();  
    string s;  
    while (getline(in, s)) if (!s.empty()) pushB(s);  
    cout << "Загружено из " << f << "\n";  
}
```

```
void clear() {  
    while (first) {  
        Node1* t = first;  
        first = first->next;  
        delete t;  
    }  
    last = nullptr;  
}  
};
```

```
struct List3 {  
    Node2* head;  
    Node2* tail;  
  
    List3() : head(nullptr), tail(nullptr) {}  
    ~List3() { clear(); }
```

```
    bool empty() const { return head == nullptr; }
```

```

void print() const {
    if (empty()) {
        cout << "Список пустой.\n";
        return;
    }
    cout << "Список: ";
    Node2* p = head;
    int i = 1;
    while (p) {
        cout << "[" << i++ << ": " << p->val << "]" ";
        p = p->next;
    }
    cout << "\n";
}

```

```

void printRev() const {
    if (empty()) {
        cout << "Список пустой.\n";
        return;
    }
    cout << "Обратный порядок: ";
    for (Node2* p = tail; p; p = p->prev) cout << "[" << p->val << "]" ";
    cout << "\n";
}

```

```

void pushF(const string& v) {
    Node2* n = new Node2(v);
    if (!head) {
        head = tail = n;
        return;
    }
    n->next = head;
    head->prev = n;
}

```

```
    head = n;
}
```

```
void pushB(const string& v) {
    Node2* n = new Node2(v);
    if (!tail) {
        head = tail = n;
        return;
    }
    tail->next = n;
    n->prev = tail;
    tail = n;
}
```

```
void addFront(int k, const string vals[]) {
    for (int i = k - 1; i >= 0; i--) pushF(vals[i]);
    cout << "Добавлено " << k << " элементов в начало.\n";
}
```

```
bool delPos(int k, const int nums[]) {
    if (empty()) {
        cout << "Список пустой.\n";
        return false;
    }
    int cnt = 0;
    for (int i = 0; i < k; i++) {
        if (delAt(nums[i])) cnt++;
    }
    if (cnt > 0) {
        cout << "Удалено " << cnt << " из " << k << " элементов.\n";
        return true;
    }
    cout << "Ничего не удалено.\n";
}
```

```
    return false;
}
```

```
bool delF() {
    if (!head) return false;
    Node2* t = head;
    if (head == tail) head = tail = nullptr;
    else {
        head = head->next;
        head->prev = nullptr;
    }
    delete t;
    return true;
}
```

```
bool delB() {
    if (!tail) return false;
    Node2* t = tail;
    if (head == tail) head = tail = nullptr;
    else {
        tail = tail->prev;
        tail->next = nullptr;
    }
    delete t;
    return true;
}
```

```
bool delAt(int pos) {
    if (pos <= 0) return false;
    Node2* p = head;
    int i = 1;
    while (p && i < pos) {
        p = p->next;
```

```

        i++;
    }
    if (!p) return false;
    if (p == head) return delF();
    if (p == tail) return delB();
    p->prev->next = p->next;
    p->next->prev = p->prev;
    delete p;
    return true;
}

```

```

void save(const string& f) const {
    ofstream out(f);
    if (!out) {
        cout << "Ошибка открытия файла.\n";
        return;
    }
    for (Node2* p = head; p; p = p->next) out << p->val << "\n";
    cout << "Сохранено в " << f << "\n";
}

```

```

void load(const string& f) {
    ifstream in(f);
    if (!in) {
        cout << "Ошибка открытия файла.\n";
        return;
    }
    clear();
    string s;
    while (getline(in, s)) if (!s.empty()) pushB(s);
    cout << "Загружено из " << f << "\n";
}

```



```

void clear() {
    while (head) {
        Node2* t = head;
        head = head->next;
        delete t;
    }
    tail = nullptr;
}
};

```

```

struct Queue {
    Node2* front;
    Node2* rear;

    Queue() : front(nullptr), rear(nullptr) {}
    ~Queue() { clear(); }

    bool empty() const { return front == nullptr; }

    void print() const {
        if (empty()) {
            cout << "Очередь пустая.\n";
            return;
        }
        cout << "Очередь: ";
        Node2* p = front;
        int i = 1;
        while (p) {
            cout << "[" << i++ << ": " << p->val << "]" ";
            p = p->next;
        }
        cout << "\n";
    }
}

```

```

void enq(const string& v) {
    Node2* n = new Node2(v);
    if (!rear) {
        front = rear = n;
        return;
    }
    rear->next = n;
    n->prev = rear;
    rear = n;
}

```

```

bool deq() {
    if (!front) return false;
    Node2* t = front;
    if (front == rear) front = rear = nullptr;
    else {
        front = front->next;
        front->prev = nullptr;
    }
    delete t;
    return true;
}

```

```

bool insBefore(int pos, const string& v) {
    if (pos <= 0) return false;
    if (pos == 1) {
        Node2* n = new Node2(v);
        if (!front) front = rear = n;
        else {
            n->next = front;
            front->prev = n;
            front = n;
        }
    }
}

```

```

    }

    return true;
}

Node2* p = front;
int i = 1;
while (p && i < pos) {
    p = p->next;
    i++;
}

if (!p) return false;
Node2* n = new Node2(v);
n->prev = p->prev;
n->next = p;
p->prev->next = n;
p->prev = n;
return true;
}

```

```

void save(const string& f) const {
    ofstream out(f);
    if (!out) {
        cout << "Ошибка открытия файла.\n";
        return;
    }
    for (Node2* p = front; p; p = p->next) out << p->val << "\n";
    cout << "Сохранено в " << f << "\n";
}

```

```

void load(const string& f) {
    ifstream in(f);
    if (!in) {
        cout << "Ошибка открытия файла.\n";
        return;
    }
}

```

```

    }

    clear();

    string s;

    while (getline(in, s)) if (!s.empty()) enq(s);

    cout << "Загружено из " << f << "\n";
}

```

```

void clear() {
    while (front) {
        Node2* t = front;
        front = front->next;
        delete t;
    }
    rear = nullptr;
}

};

```

```

struct Stack {
    Node1* top;

    Stack() : top(nullptr) {}
    ~Stack() { clear(); }

    bool empty() const { return top == nullptr; }

    void print() const {
        if (empty()) {
            cout << "Стек пустой.\n";
            return;
        }

        cout << "Стек: ";
        Node1* p = top;
        int i = 1;
    }
}

```

```

while (p) {
    cout << "[" << i++ << ": " << p->val << "]" ";
    p = p->next;
}
cout << "\n";
}

```

```

void push(const string& v) {
    Node1* n = new Node1(v);
    n->next = top;
    top = n;
}

```

```

bool pop() {
    if (!top) return false;
    Node1* t = top;
    top = top->next;
    delete t;
    return true;
}

```

```

void save(const string& f) const {
    ofstream out(f);
    if (!out) {
        cout << "Ошибка открытия файла.\n";
        return;
    }
    for (Node1* p = top; p; p = p->next) out << p->val << "\n";
    cout << "Сохранено в " << f << "\n";
}

```

```

void load(const string& f) {
    ifstream in(f);

```

```
if (!in) {  
    cout << "Ошибка открытия файла.\n";  
    return;  
}  
clear();
```

```
Node1* first = nullptr;  
Node1* last = nullptr;  
string s;
```

```
while (getline(in, s)) {  
    if (s.empty()) continue;  
    Node1* n = new Node1(s);  
    if (!first) first = last = n;  
    else {  
        last->next = n;  
        last = n;  
    }  
}
```

```
while (first) {  
    Node1* prev = nullptr;  
    Node1* cur = first;  
    while (cur->next) {  
        prev = cur;  
        cur = cur->next;  
    }  
    push(cur->val);  
    if (!prev) {  
        delete first;  
        first = nullptr;  
    }  
    else {
```

```

        delete cur;

        prev->next = nullptr;
    }
}

cout << "Загружено из " << f << "\n";
}

void clear() {
    while (top) {
        Node1* t = top;

        top = top->next;

        delete t;
    }
}

};

void demo1() {
    cout << "\n ОДНОНАПРАВЛЕННЫЙ СПИСОК \n";

    List1 L;

    L.pushB("A"); L.pushB("B"); L.pushB("C"); L.pushB("D"); L.pushB("E");

    cout << "Исходный: "; L.print();

    string a[] = { "X", "Y", "Z" };

    L.addFront(3, a);

    cout << "После добавления: "; L.print();

    int p[] = { 2, 4, 6 };

    L.delPos(3, p);

    cout << "После удаления: "; L.print();

    L.save("list1.txt");

    L.clear();
}

```

```
L.load("list1.txt");  
L.print();  
}
```

```
void demo2() {  
    cout << "\n ДВУНАПРАВЛЕННЫЙ СПИСОК \n";  
    List2 L;  
    L.pushB("Red"); L.pushB("Green"); L.pushB("Blue"); L.pushB("Yellow"); L.pushB("Black");  
    cout << "Исходный: "; L.print();  
  
    string a[] = { "First", "Second", "Third" };  
    L.addFront(3, a);  
    cout << "После добавления: "; L.print();  
  
    int p[] = { 1, 3, 5 };  
    L.delPos(3, p);  
    cout << "После удаления: "; L.print();  
  
    L.save("list2.txt");  
    L.clear();  
    L.load("list2.txt");  
    L.print();  
}
```

```
void demo3() {  
    cout << "\n ДВУСВЯЗНЫЙ СПИСОК \n";  
    List3 L;  
    L.pushB("One"); L.pushB("Two"); L.pushB("Three"); L.pushB("Four"); L.pushB("Five");  
    cout << "Исходный: "; L.print();  
  
    string a[] = { "Start1", "Start2", "Start3" };  
    L.addFront(3, a);  
    cout << "После добавления: "; L.print();  
}
```



```

int p[] = { 2, 4, 6 };

L.delPos(3, p);

cout << "После удаления: "; L.print();

L.printRev();


L.save("list3.txt");

L.clear();

L.load("list3.txt");

L.print();
}


void demo4() {

    cout << "\n ОЧЕРЕДЬ \n";

    Queue Q;

    Q.enq("Q1"); Q.enq("Q2"); Q.enq("Q3"); Q.enq("Q4"); Q.enq("Q5");

    cout << "Исходная: "; Q.print();


    Q.insBefore(2, "P1");

    Q.insBefore(4, "P2");

    cout << "После вставки: "; Q.print();


    Q.save("queue.txt");

    Q.clear();

    Q.load("queue.txt");

    Q.print();
}


void demo5() {

    cout << "\n СТЕК \n";

    Stack S;

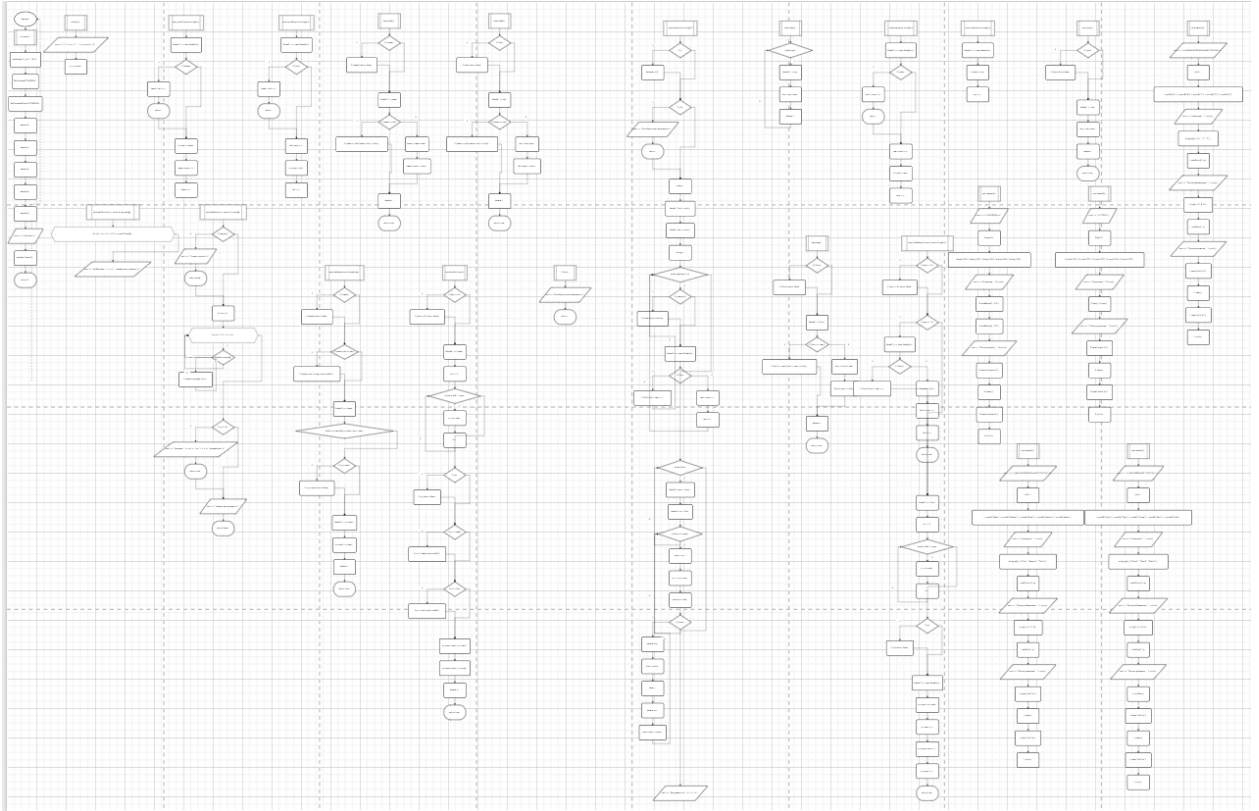
    S.push("S1"); S.push("S2"); S.push("S3"); S.push("S4"); S.push("S5");

    cout << "Исходный: "; S.print();
}

```

```
S.pop(); S.pop();  
cout << "После удаления: "; S.print();  
  
S.save("stack.txt");  
S.clear();  
S.load("stack.txt");  
S.print();  
}  
  
int main() {  
    setlocale(LC_ALL, "RU");  
    SetConsoleCP(65001);  
    SetConsoleOutputCP(65001);  
  
    demo1();  
    demo2();  
    demo3();  
    demo4();  
    demo5();  
  
    cout << "\nГотово.\n";  
    system("pause");  
    return 0;  
}
```

Блок-схема



Вывод программы

```

ОДНОНАПРАВЛЕННЫЙ СПИСОК
Исходный: Список: [1: A] [2: B] [3: C] [4: D] [5: E]
Добавлено 3 элементов в начало.
После добавления: Список: [1: X] [2: Y] [3: Z] [4: A] [5: B] [6: C] [7: D] [8: E]
Удалено 3 из 3 элементов.
После удаления: Список: [1: X] [2: Z] [3: A] [4: C] [5: D]
Сохранено в list1.txt
Загружено из list1.txt
Список: [1: X] [2: Z] [3: A] [4: C] [5: D]

ДВУНАПРАВЛЕННЫЙ СПИСОК
Исходный: Список: [1: Red] [2: Green] [3: Blue] [4: Yellow] [5: Black]
Добавлено 3 элементов в начало.
После добавления: Список: [1: First] [2: Second] [3: Third] [4: Red] [5: Green] [6: Blue] [7: Yellow] [8: Black]
Удалено 3 из 3 элементов.
После удаления: Список: [1: Second] [2: Third] [3: Green] [4: Blue] [5: Black]
Сохранено в list2.txt
Загружено из list2.txt
Список: [1: Second] [2: Third] [3: Green] [4: Blue] [5: Black]

ДВУСВЯЗНЫЙ СПИСОК
Исходный: Список: [1: One] [2: Two] [3: Three] [4: Four] [5: Five]
Добавлено 3 элементов в начало.
После добавления: Список: [1: Start1] [2: Start2] [3: Start3] [4: One] [5: Two] [6: Three] [7: Four] [8: Five]
Удалено 3 из 3 элементов.
После удаления: Список: [1: Start1] [2: Start3] [3: One] [4: Three] [5: Four]
Обратный порядок: [Four] [Three] [One] [Start3] [Start1]
Сохранено в list3.txt
Загружено из list3.txt
Список: [1: Start1] [2: Start3] [3: One] [4: Three] [5: Four]

ОЧЕРЕДЬ
Исходная: Очередь: [1: Q1] [2: Q2] [3: Q3] [4: Q4] [5: Q5]
После вставки: Очередь: [1: Q1] [2: P1] [3: Q2] [4: P2] [5: Q3] [6: Q4] [7: Q5]
Сохранено в queue.txt
Загружено из queue.txt
Очередь: [1: Q1] [2: P1] [3: Q2] [4: P2] [5: Q3] [6: Q4] [7: Q5]

СТЕК
Исходный: Стек: [1: S5] [2: S4] [3: S3] [4: S2] [5: S1]
После удаления: Стек: [1: S3] [2: S2] [3: S1]
Сохранено в stack.txt
Загружено из stack.txt
Стек: [1: S3] [2: S2] [3: S1]

```